

AI-Powered Document & Certificate Forgery Detection

Architecture Review, Tech Stack Opinion & Workflow — Java / Spring Boot Edition

My Opinion On Your Tech Stack

Verdict: Yes — this is a solid, resume-appropriate stack, with one adjustment. Spring Boot + MySQL + React/HTML-CSS-JS is a completely reasonable choice for the application layer. Since Java is your core language for your resume, the forensic/AI modules (OCR, image analysis, metadata, QR) should also be implemented as Java libraries rather than a separate Python service — this keeps the entire project genuinely "Java-built" end to end.

- **MySQL — Good fit.**
Relational structure suits this project well: documents, scan reports, flagged regions, users, and audit logs all have clear relationships. Use Spring Data JPA on top of it.
- **Spring Boot — Good fit — and reuse what you know.**
You already used Spring Boot for VeriShield AI. Reusing it here means less setup overhead and a stronger "I can build production-style Java backends" story across two projects on your resume.
- **HTML/CSS/JS or React — Either works — lean toward React.**
React is worth it here specifically because the report UI is dynamic: score gauges, a canvas overlay highlighting tampered regions, live upload progress. Plain JS is fine if you want a faster, simpler build.
- **Java for the AI/forensics layer — Needs specific libraries — not raw Java.**
Java has no first-class OCR/CV ecosystem like Python's, but mature bindings exist: Tess4J (OCR), OpenCV via JavaCV (image analysis), metadata-extractor (EXIF forensics), and ZXing (QR codes). These are real, widely-used libraries — not a compromise.

Bottom line

Keep your stack exactly as you listed it. Build one Spring Boot application (not a Python microservice) with dedicated service classes for OCR, image forensics, metadata analysis, QR verification, and score aggregation. This is what the rest of this document is architected around.

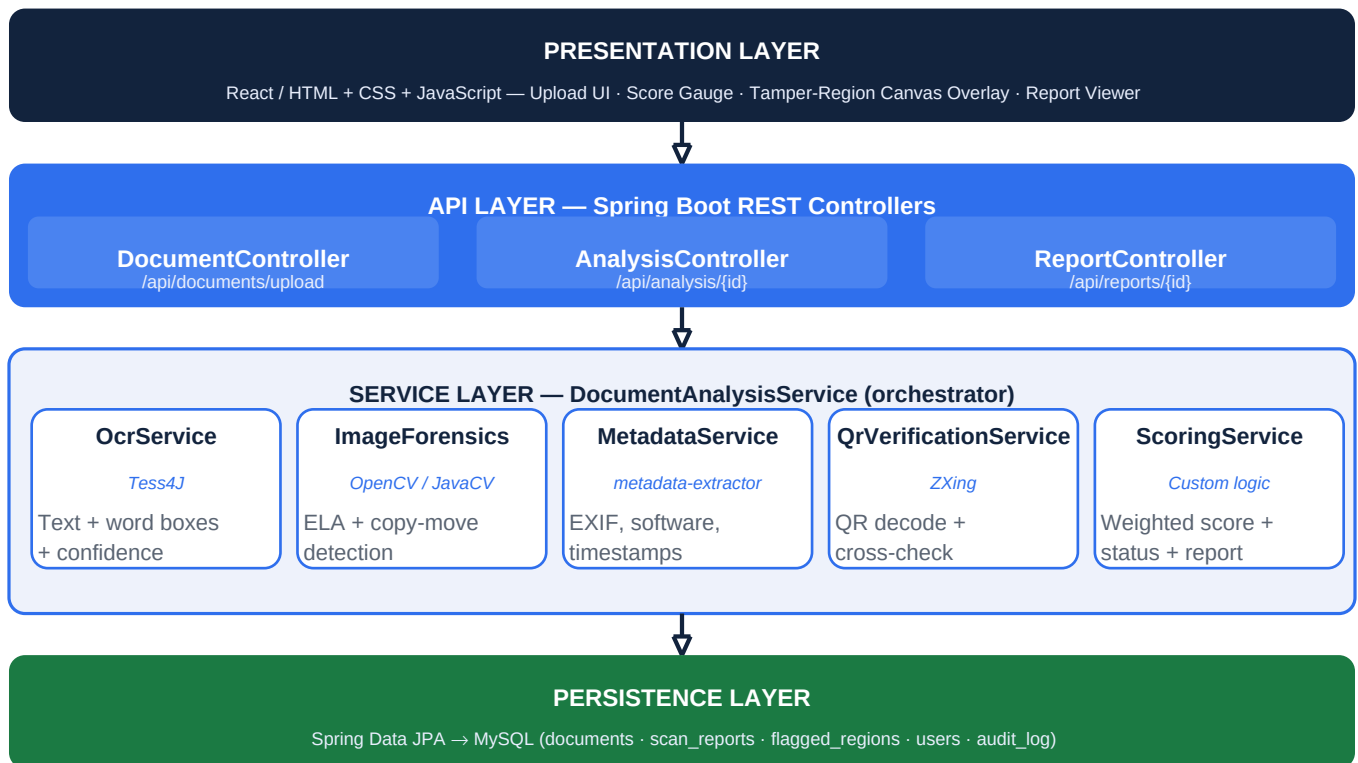
Finalized Tech Stack (Full Java Build)

Layer	Technology	Purpose
Frontend	React (or HTML + CSS + JavaScript)	Upload UI, authenticity score display, tamper-region overlay, report view
Backend / API	Spring Boot (Java 17+), Spring Web, Spring Data JPA	REST controllers, orchestration, business logic, security
Database	MySQL 8	Stores documents metadata, scan reports, flagged regions, users, audit trail
OCR Engine	Tess4J (Java JNA wrapper for Tesseract OCR)	Extracts text + word bounding boxes + confidence scores
Image Forensics	OpenCV via JavaCV / org.openpnp:opencv	Error Level Analysis (ELA), copy-move detection, tamper region localization
Metadata Forensics	metadata-extractor (com.drewnoakes)	EXIF data: editing software used, creation/modified timestamps, device info
QR Verification	ZXing ("Zebra Crossing")	Detects & decodes embedded QR codes, cross-checks against document fields
Deep Learning (optional/advanced)	DJL — Deep Java Library (Amazon), running pretrained ONNX models	CNN/ResNet/ViT-based tamper classification, without leaving the JVM
File Storage	Local filesystem (dev) → AWS S3 / MinIO (production)	Stores uploaded originals + generated tamper-highlighted images
Build & Deploy	Maven/Gradle, Docker, Docker Compose (Spring Boot + MySQL)	Single reproducible deployment, good for demo & viva

Note: The DJL/ONNX deep-learning row is marked optional — start with the rule-based / classical CV forensic techniques (ELA, copy-move, metadata, OCR text analysis) since those are explainable and fast to build. Add the CNN classifier later as a stretch goal once the base pipeline works end-to-end — that phased approach also reads well in a viva.

System Architecture

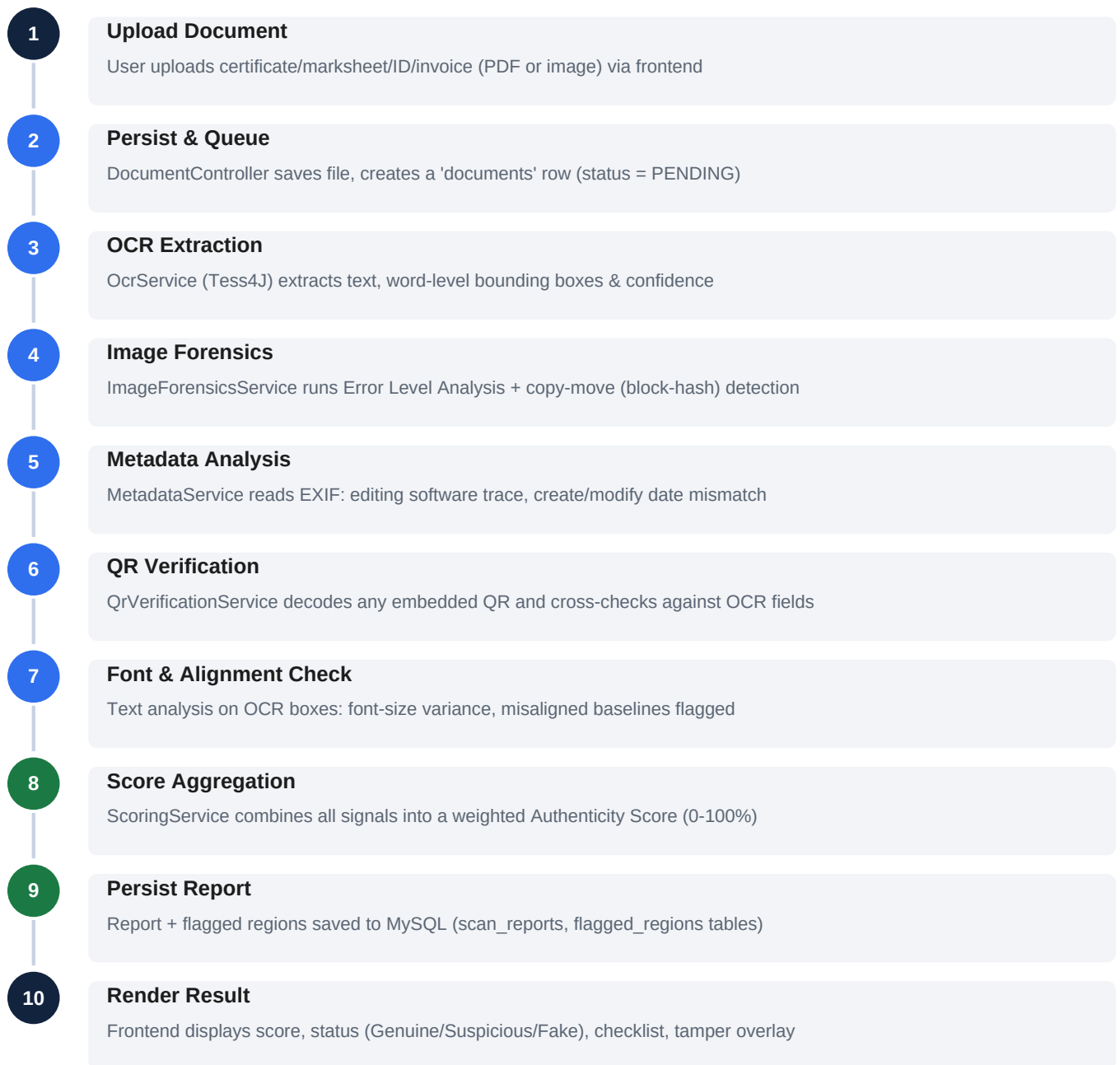
Layered Java architecture — single Spring Boot application



Uploaded files are stored on disk (or S3/MinIO in production) and referenced by path/URL from the **documents** table — only metadata and analysis results go into MySQL, keeping the database lean.

End-to-End Workflow

What happens between upload and report



MySQL Schema Sketch & Module Notes

Core Tables

documents

id, original_filename, storage_path, doc_type, uploaded_by, uploaded_at, status

scan_reports

id, document_id (FK), authenticity_score, status, ocr_text, risk_level, created_at

flagged_regions

id, scan_report_id (FK), region_type, x, y, width, height, reason

metadata_findings

id, scan_report_id (FK), software_used, created_date, modified_date, is_suspicious

qr_verifications

id, scan_report_id (FK), qr_present, decoded_data, matches_document_fields

users

id, name, email, role, password_hash, created_at

Key Implementation Notes

- **Copy-move detection** without training data: split the image into overlapping blocks, hash each block (perceptual/average hash), and flag any two non-adjacent blocks with near-identical hashes as a likely copy-pasted region. Fully implementable with OpenCV Java + a simple hashing routine.
- **Error Level Analysis (ELA)**: re-save the image at a known JPEG quality, diff it against the original, and amplify the result — genuine, unedited regions show uniform low error; edited regions (pasted text, altered numbers) stand out brighter.
- **Font/alignment inconsistency**: use Tess4J's word-level bounding boxes to compute the variance in character height and baseline Y-position across the document — outliers indicate edited fields.
- **Scoring model**: start with a transparent weighted rule (e.g., OCR confidence 15%, ELA 25%, copy-move 25%, metadata 15%, font/alignment 10%, QR match 10%) before considering a learned model — explainable scoring is easier to defend in a viva than a black-box one.
- **DJL/ONNX (optional)**: if you want a genuine CNN in the loop, export a pretrained tamper-detection model to ONNX and run inference via DJL — this stays 100% inside the Java/Spring Boot process.